

Vectorization path planning for autonomous mobile agent in unknown environment

Qi-Lei Xu · Gong-You Tang

Received: 13 June 2012 / Accepted: 7 September 2012
© Springer-Verlag London Limited 2012

Abstract This paper presents a novel real-time path planning algorithm, called InsertBug, for an autonomous mobile agent in completely unknown environment. By using the algorithm, all the planned paths are described and stored by vectors. The algorithm combines range sensor data with safety radius to determine the blocking obstacles and calculate the shortest path by choosing the intermediate points. When there is obstacle blocking the current path, the intermediate points will be calculated, and the planned path will be regenerated by inserting the intermediate points. The local optimum avoidance strategy is also considered in this algorithm by specifying a fixed direction. The agent will return to the optimal direction after running out of the local optimum. Different simulation parameters are taken to show the advantages of this algorithm. Moreover, the performance of this algorithm has also been evaluated by comparing with another usual method via simulations. The results show that the safety performance and time requirements of this algorithm are significant superior to the algorithm contrasted with.

Keywords Unknown environment · Path planning · Vectorization path · Bug algorithm

1 Introduction

Path planning is a significant issue in autonomous mobile agent navigation researches. Path planning aims at finding feasible and free-collision paths from the start position of

the agent to its goal position. Meanwhile, the planning paths should satisfy some optimal performance indexes, such as time optimal, minimum energy consumption, and shortest path length, etc [1–3].

Recently, researchers pay more attention to path planning in completely unknown environment. In this case, each agent uses its own sensor to obtain the limited environment information and then completes its path planning task independently. Common methods include artificial potential field method [4–7], fuzzy logic algorithm [1, 8–10], and the Bug algorithms [11–16], etc.

The Bug algorithm family is well-known robot navigation algorithms with proven termination conditions for unknown environment [11]. Bug1 and Bug2 [12], VisBug [13], DistBug [14], and TangentBug [15] are the most commonly used and referred types of Bug algorithm. All the Bug algorithms try to get shorter path planning result, shorter timing, better performance, and simpler calculation. In their researches, the robot moves toward the goal position with a contact sensor [12] or a range sensor [13–16] to detect the boundary of obstacles. When the robot encounters the obstacles, it circumnavigates the obstacles until the motion toward the goal position is allowable again. The TangentBug calculates a shorter path to the goal position by selecting the optimal circulatory direction and leaving obstacles' boundary as soon as possible. The disadvantage of the TangentBug is that its computational burden is heavy because of the robot being required to scan 360° detecting range before calculating the real-time local tangent graph (LTG) of the obstacles and making the decision to move to the next position. Latest type Bug algorithm is EgressBug [16]. In EgressBug, two simple algorithms were presented to indicate robots' moving. PointBug and EgressBug both produce shorter path than TangentBug algorithm. But all of the Bug algorithms make the robots move along obstacles'

Q.-L. Xu (✉) · G.-Y. Tang
College of Information Science and Engineering,
Ocean University of China, Qingdao 266100, China
e-mail: candy_hxu@sina.com

boundary without considering the safety radius of the moving robots, so their security needs to be enhanced.

This paper presents a vectorization path planning approach called InsertBug for the omni-directional autonomous mobile agent in a completely unknown environment. The agent computes its own path and regenerates the path by using its own sensor data to achieve the goal position with free-collision. Firstly, according to the start position and the goal position, generate and save the vectorization initial path. Then, the agent moves to the goal position along the planned path with two actions that are turning to the corresponding angle following the planned path and straightly moving to the prescribed distance. The two actions are repeated when the agent achieves the turning point. During the moving process, the agent samples and analyzes the environmental information at constant intervals. When the agent encounters an obstacle, it will calculate and select the some feasible intermediate points correspondingly according to its sensor data and the pre-set rules. The selected points will be inserted into the current path to regenerate the free-collision path. The path recorded or stored in the form of vector can reduce the requirement of storage space and, to a certain extent, simplify the control of the agent. Taking the safety radius between the agent and the obstacle into our consideration during the moving process can increase the practicality and the security of the algorithm.

The paper is organized as follows: in Sect. 2, we present our solution approach and outline the overall structure of the vectorization path planning method. In Sect. 3, we present the experimental results to show the satisfactory performance of our approach in completely unknown environment. Finally, the conclusion is presented in Sect. 4.

2 Insertbug algorithm

2.1 Vectorization path description

In this paper, we consider the path planning for omni-directional autonomous mobile agent in maps of broad scope, so the agent can be assumed to be a point with no positioning error. Agent's range sensor is assumed to provide perfect readings of the distance to the obstacles within the radius of the maximum detecting range. We solve the problem in the configuration spaces [17]. The start and the goal are labeled q_{start} and q_{goal} , respectively.

We make the agent detect the circum obstacle information when it moves with a certain distance. After finish the detection of the current path section, the agent will immediately detect the environment when it steers to the next section. In InsertBug algorithm, all the path sections

are described by vectors, and the former vector's terminal point is the latter's starting point. Based on the sensed obstacle information and the current moving path section, the agent determines the obstacle blocking or not. If there are obstacles blocking the current path vector set, and the current path vector set will be regenerated to the new path by calculating and inserting an intermediate point into the current path.

$\mathbf{P}_i$ denotes any i th planned path vector set, and in the set, the former vector's terminal point is the latter's starting point,

$$\mathbf{P}_i = \alpha_{i1}, \alpha_{i2}, \dots, \alpha_{in} \quad (1)$$

where $\alpha_{ik} (k = 1, 2, \dots, n)$ is the k th vector in the path vector set $\mathbf{P}_i$, the terminal point of vector $\alpha_{i(k-1)}$ is the start point of vector α_{ik} , $\alpha_{ik} = (\theta_{ik}, \|\alpha_{ik}\|)$, θ_{ik} is the angle between the vector α_{ik} and $\alpha_{i(k-1)}$, and $-\pi \leq \theta_{ik} \leq \pi$. The clockwise direction is positive, and the counterclockwise direction is negative. $\|\alpha_{ik}\|$ is the vector module, which is the distance the agent should move after it turns θ_{ik} .

In this paper, following steps are taken to solve path planning. Firstly, build the initial path vector set. Secondly, determine whether or not there is obstacle blocking the current path vector. When there is obstacle blocking, calculate the intermediate point. And then regenerate the path vector set by inserting the intermediate point into the current path vector set.

2.2 Obstacle determination

In order to get the no collision path, we make the agent determine the obstacle blocking in its sensor range at constant intervals during the moving process. The following rule is presented to determine the obstacle.

Rule 1: If the two conditions hold:

- (i) $d \cdot \sin|\varphi| < R_{\text{safe}}$,
- (ii) $d \leq d_r - R_{\text{safe}}$,

the obstacle obstructs the current path, then the path needs to be regenerated by inserting an intermediate point; otherwise, the path does not need to regenerate, where d_r is the current path remnant distance, d is the distance between the agent and sensed obstacle, R_{safe} is the safety radius, and φ is the angle between two vectors: (1) one that starts at agent's current position and terminates at the obstacle point and (2) the other as current path vector.

Condition (i) ensures that the agent keep a safety distance away to the obstacles.

During the moving process, the single obstacle point or more will be detected. If there are more than one obstacle points detected, every obstacle point will be calculated according to *Rule 1* to determine whether it will block the current path.

2.3 Intermediate point calculation

When there are obstacles blocking the current path, the agent calculates the shortest distance $d_{\min}$, and then calculates the intermediate point in accordance with the following rules. Taking the shortest obstacle distance among the distances in accord with the *Rule 1*, it can be inferred from *Rule 1* that $d_{\min} \geq R_{\text{safe}}$.

Rule 2: Taking the direction of $d_{\min}$ as the reference direction, rotate angle θ to generate a vector with the module to be $d_{\min}$, and the start point as the current position. If there exists a vector making the three conditions hold:

- (i) The terminal point of the vector is not on the obstacles,
- (ii) $d_{\min} \cdot \sin|\theta| \geq R_{\text{safe}}$,
- (iii) m takes minimum value,

the vector can be added into the path vector set, and the terminal point of the vector is the intermediate point, where $\theta = \pm m \cdot \Delta\theta$, $m = 1, 2, \dots, \lfloor \pi/\Delta\theta \rfloor$.

Condition (i) and (ii) ensure the agent move to the new calculated intermediate point without collision in static environment. Condition (iii) ensures the optimality of the planned path.

Figure 1 shows the calculation of the intermediate point. To further explain, we take semicircle to replace the detecting circle area. The intersection point of the straight line of the semicircle and the path is the current position of agent. The initial path starts at position q_{start} and ends at position q_{goal} . The agent moves along the initial path and detects the environment at a constant interval. When the agent reaches position q_0 , one obstacle point is detected. The path does not need to be regenerated, because the obstacle point does not meet the *Rule 1*, and the agent still moves along the original path. When the agent reaches position q_1 , there are obstacle points meeting *Rule 1*. The intermediate point $q_{\text{insert}1}$ is then calculated in accordance with the *Rule 2*, and the path is regenerated. Agent moves along the regenerated path, repeating to detect the environment and calculate the obstacle point with *Rule 1*. When it reaches the position $q_{\text{insert}1}$ and turns to the next path direction, there are obstacle points meet *Rule 1*. The intermediate point $q_{\text{insert}2}$ is calculated, and the path is regenerated again.

2.4 Path regeneration

As shown in Fig. 2a, the initial path vector set $\mathbf{P}_0$ only contains one vector formed by the line connecting start

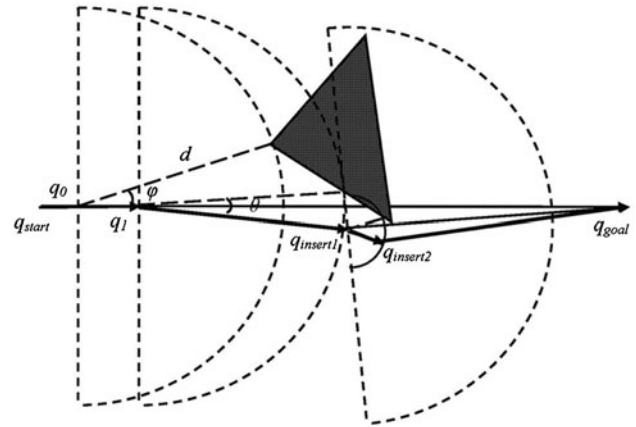


Fig. 1 Obstacle determination and intermediate point calculation

position q_{start} with goal position q_{goal} . The angle of the vector is zero. (If the agent is not facing the goal position, steer the agent to the goal position in its original place.)

$$\mathbf{P}_0 = \alpha_{01} \quad (2)$$

$$\alpha_{01} = (0, \|\alpha_{01}\|) \quad (3)$$

Assuming *Obstacle1* blocked the path, we add intermediate point $q_{\text{insert}1}$ into initial path vector set $\mathbf{P}_0$ to get the regenerated path vector set $\mathbf{P}_1$,

$$\mathbf{P}_1 = \alpha_{11}, \alpha_{12} \quad (4)$$

$$\alpha_{11} = (\theta_{11}, \|\alpha_{11}\|) \quad (5)$$

$$\alpha_{12} = (\theta_{12}, \|\alpha_{12}\|) \quad (6)$$

where α_{11} and α_{01} are known. α_{12} and θ_{12} can be calculated based on the Cosine theorem,

$$\|\alpha_{12}\| = \sqrt{\|\alpha_{01}\|^2 + \|\alpha_{11}\|^2 - 2\|\alpha_{01}\| \cdot \|\alpha_{11}\| \cdot \cos \theta_{11}} \quad (7)$$

$$\theta_{12} = \cos^{-1}((\|\alpha_{01}\| \cdot \cos \theta_{11} - \|\alpha_{11}\|)/\|\alpha_{12}\|) \quad (8)$$

As shown in Fig. 2b, the agent keeps moving in accordance with path vector set $\mathbf{P}_1$. During the moving process along path vector α_{12} , *Obstacle2* will block the path. We add intermediate point $q_{\text{insert}2}$ into the path vector set $\mathbf{P}_1$ to get the regenerated path vector set $\mathbf{P}_2$,

$$\mathbf{P}_2 = \alpha_{21}, \alpha_{22}, \alpha_{23}, \alpha_{24} \quad (9)$$

$$\alpha_{2k} = (\theta_{2k}, \|\alpha_{2k}\|), k = 1, 2, 3, 4 \quad (10)$$

where $\theta_{22} = \theta_{12}$, α_{23} , and $\|\alpha_{22}\|$ is known. α_{24} and θ_{24} can be calculated based on the Cosines theorem,

$$\|\alpha_{24}\| = \sqrt{(\|\alpha_{12}\| - \|\alpha_{22}\|)^2 + \|\alpha_{23}\|^2 - 2(\|\alpha_{12}\| - \|\alpha_{22}\|) \cdot \|\alpha_{23}\| \cdot \cos \theta_{23}} \quad (11)$$

Fig. 2 Path vector set regeneration **a** path vector set P_1 , **b** path vector set P_2

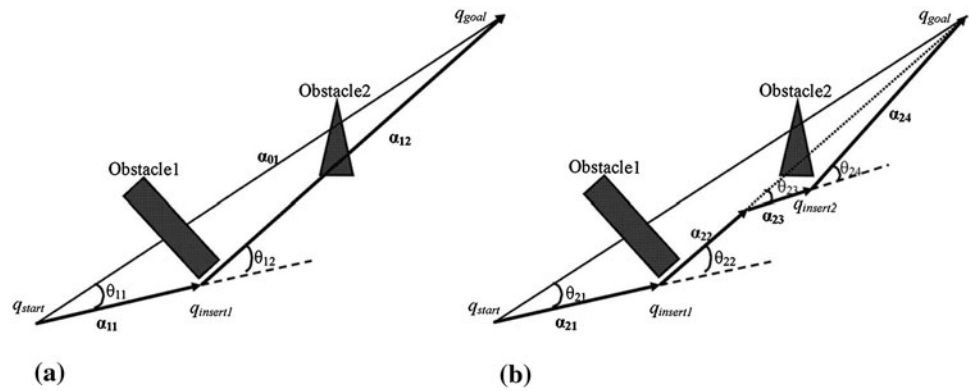
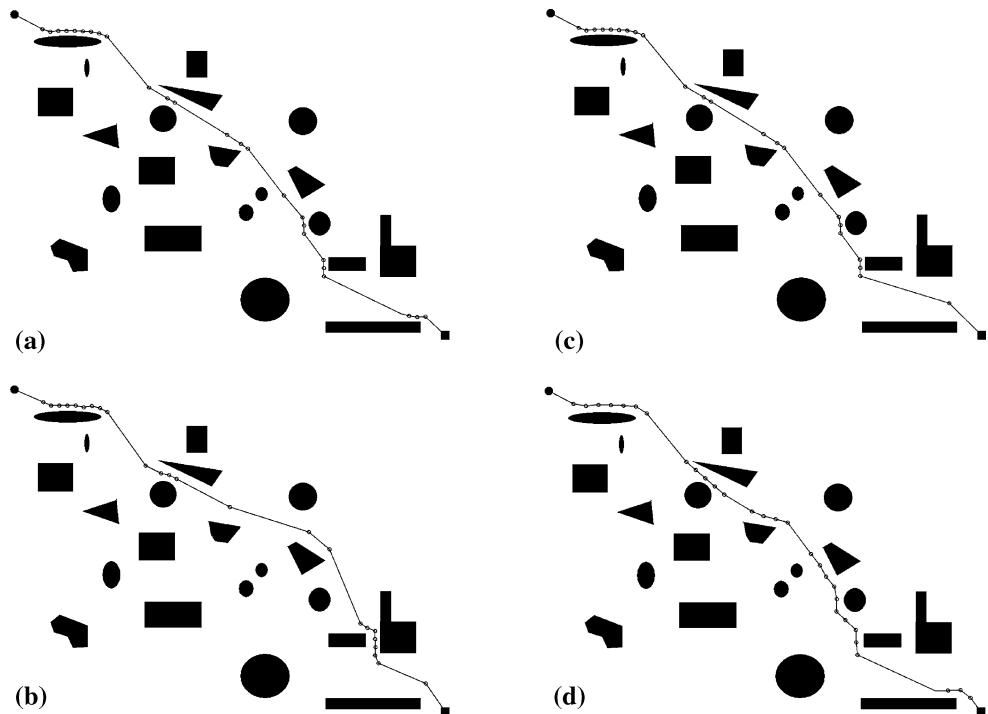


Fig. 3 Simulation results of different parameters in the same map **a** parameter 1, **b** parameter 2, **c** parameter 3, **d** parameter 4



$$\theta_{24} = \cos^{-1}(((\|\alpha_{12}\| - \|\alpha_{22}\|) \cdot \cos \theta_{23} - \|\alpha_{23}\|) / \|\alpha_{24}\|) \quad (12)$$

Similarly, we add intermediate point $q_{inserti}$ into path vector set P_i to get regenerated path vector set P_{i+1} ,

$$P_{i+1} = \alpha_{(i+1)1}, \alpha_{(i+1)2}, \dots, \alpha_{(i+1)m} \quad (13)$$

$$\alpha_{(i+1)l} = (\theta_{(i+1)l}, \|\alpha_{(i+1)l}\|), l = 1, 2, \dots, m \quad (14)$$

The moving direction and distance can be calculated according to the Cosines theorem and the path information before regeneration.

2.5 Local optimum avoidance strategy

If the path is planned only according to the two rules mentioned above, the solution will be locally optimized when the agent's

moving direction is consistent with the centerline of the concave obstacle. As a result, the planning will fail. Following *Rule 3* is presented to determine the local optimum.

Rule 3: If regenerated path vector set P_i makes following conditions hold:

- (i) $\|\alpha_{i(n-1)}\| = \|\alpha_{i(n-2)}\| = R_{safe}$,
- (ii) $|\theta_{i(n-1)}| = \pi$,

the current planning is believed to be fallen into the local optimum.

After the local optimum determination, the intermediate point calculation should be based on *Rule 4* rather than *Rule 2* to step out of the local optimum. The *Rule 4* is described below.

Rule 4: Taking the direction of d_{min} as the reference direction, rotate angle θ to generate a vector with the

module to be $d_{\min}$, and the start point as the current position. If there exists a vector making the three conditions hold:

- (i) The terminal point of the vector is not on the obstacles,
- (ii) $d_{\min} \cdot \sin|\theta| \geq R_{\text{safe}}$,
- (iii) m takes minimum value,

the vector can be added into the path vector set, and the terminal point of the vector is the intermediate point, where $\theta = m \cdot \Delta\theta$, $m = 1, 2, \dots, \lfloor \pi/\Delta\theta \rfloor$.

In the *Rule 2*, the angle θ could be either positive or negative to get the optimal path, which indicating the agent could change its moving direction in either clockwise or counterclockwise direction. However, based on *Rule 4*, the agent changes its moving direction only in clockwise direction to run out of the local optimum.

To ensure the agent to get optimal planning result after running out of the local optimum, we present *Rule 5* to determine whether the agent should return to *Rule 2*.

Rule 5: If the following conditions hold:

- (i) $\|\alpha_{\text{in}}\| \geq \|\alpha_{(i+1)m}\|$,
- (ii) $|\theta| < \pi/2$,

the current planning is believed to have already run out of the local optimum, and the next calculation should use *Rule 2* again, where α_{in} is the path vector in un-regenerated path vector set, $\alpha_{(i+1)m}$ is the path vector in regenerated path vector set.

3 Simulation results

In order to prove the feasibility and the optimality of the proposed algorithm, we run simulation software, programmed in C#, on a notebook with i5-M430 (2.27 GHz) CPU and 2GB memory. Figure 3 shows the results of using different planning parameters in the same map. The map size is $1,024 \times 768$ pixel with each pixel represents one unit length. We use the map to simulate the unknown actual environment. The initial distance between the start position and goal position is 1,250.61. In Fig. 3, filled circle represents the start position, filled square represents

the goal position, and open circle represents the inserted intermediate point. The entities are obstacles, and the solid line is the moving path of the agent.

Table 1 shows the data of Fig. 3. It is assumed the moving and distance detecting time is zero. Therefore, the planning time in Table 1 is all the time cost during the path planning, and the average time is the time cost for calculating each intermediate point. To calculate each intermediate point, we need to calculate the angle $\theta = \pm m \cdot \Delta\theta$ (where m is the number of iterations) which related to the distance and angle between obstacle and agent. As a result, the total time cost is not completely proportional to the number of intermediate point. It is indicated from Table 1 and Fig. 3 that agent can move closer to obstacle's contour by reducing steering angle $\Delta\theta$, when the detecting radius and safety radius are constant. The path length is reduced, whereas the number of intermediate point is increased. When the detecting radius and steering angle are constant, agent can also move closer to obstacle's contour by

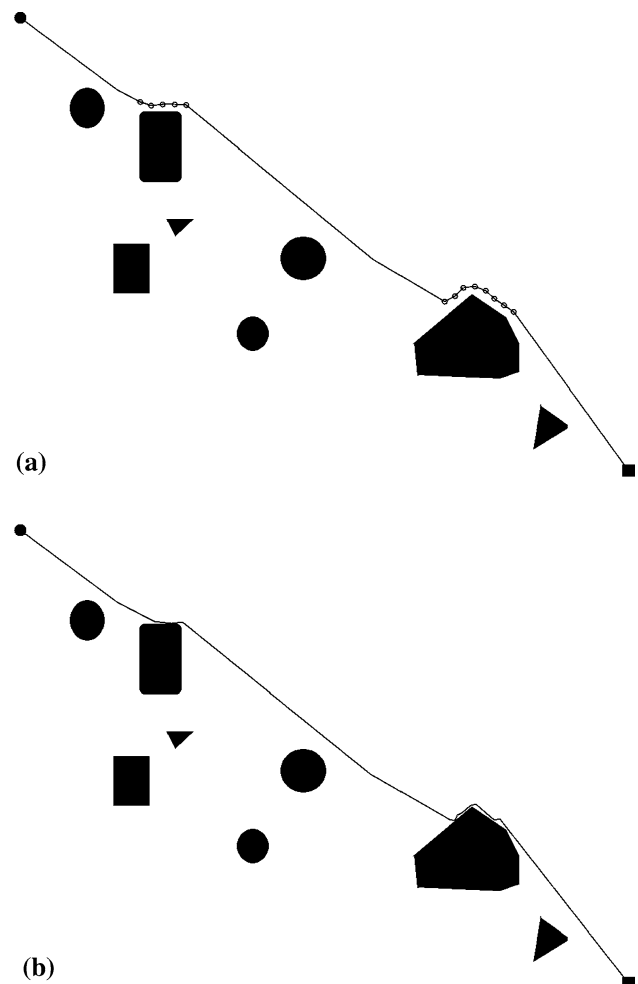


Fig. 4 Simulation results of map 1 **a** InsertBug, **b** TangentBug

Table 1 Result contrast of different parameters in the same map

Map	D	R_{safe}	$\Delta\theta$ ($^\circ$)	Intermediate point number	Path length (pixels)	Planning time (ms)	Average time (ms)
(a)	200	10	9	25	1,330.45	7.03	0.28
(b)	200	15	9	29	1,355.45	10.09	0.35
(c)	300	10	9	23	1,328.62	7.87	0.34
(d)	200	10	12	24	1,355.65	8.93	0.37

Fig. 5 Simulation results of map 2 **a** InsertBug, **b** TangentBug

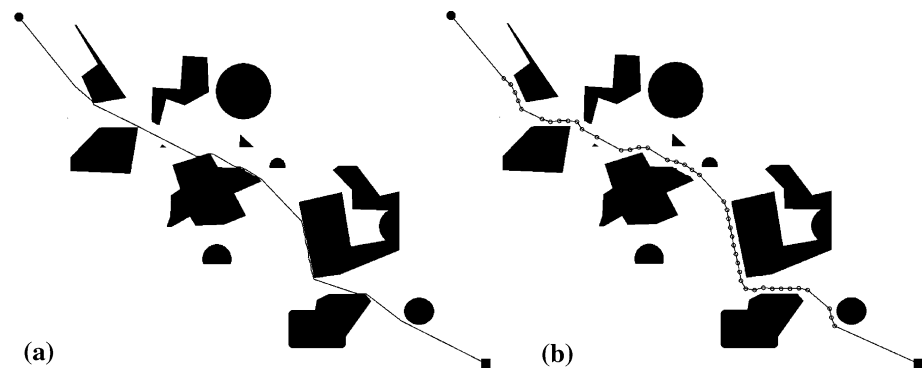


Table 2 Planning results with different algorithms

Algorithm	Map	Path length	Planning time (ms)
InsertBug	1	1,313.91	4.93
TangentBug	1	1,312.06	19.49
InsertBug	2	1,381.20	7.31
TangentBug	2	1,321.56	54.47

reducing the safety radius. In this case, path length and intermediate point number are both decreased. When safety radius and steering angle are constant, increasing detecting radius can make the agent percept the obstacle in advance, accordingly reduce the computation of the intermediate points and shorten the path planning result. Therefore, in practical applications, we suggest to try to increase the detecting radius, reduce safety radius, and steering angle. In a special case containing a narrow channel, the safety radius cannot be too large, otherwise the agent will not able to move through the narrow channel. Simulation was performed in plenty of maps, and all of the agents can achieve the goal position without collision. The average time cost for calculating the intermediate points is 0.5 ms, proving that the real-time performance of the proposed method meets the requirements.

The simulations compare InsertBug with TangentBug in two maps. In these simulations, detecting radius $D = 300$, safety radius $R_{\text{safe}} = 10$, steering angle $\Delta\theta = 9^\circ$, and the sensor range of TangentBug is 300. The planning results are shown in Figs. 4, and 5 with the specific data given in Table 2.

From Figs. 4, 5 and Table 2, it is found that the path planned by TangentBug is closer to the obstacle contour, whereas the agent and obstacles can keep a certain safety distance away from each other in the proposed algorithm. Consequently, the path planned by the proposed algorithm is longer than TangentBug, but its safety performance is superior to TangentBug. In addition, TangentBug requires continuously computing the LTG of obstacles in its 360° sensor range, which makes its computation to be more

complex than our algorithm. Therefore, the proposed algorithm surpasses TangentBug in the planning time cost, and more complex the obstacles are, more obvious the superiority is.

4 Conclusions

The InsertBug is presented in this paper. In this algorithm, the planned paths are described and stored in the form of vectorization. The path planning result is easy to execute, and the requirement of storage space is decreased especially in large area and complex environment. We fully considered the safety distance between agent and obstacles to improve the practicality and safety of the algorithm. The presented strategy for selecting intermediate points only requires a simple calculation, and it is beneficial for agents to regenerate its path in real-time. It is proved that the proposed algorithm can achieve the path planning for agents in completely unknown environment.

Acknowledgments This work was supported by the National Natural Science Foundation of China (61074092) and National Natural Science Foundation of Shandong Province (ZR2010FM019).

References

1. Tarokh M (2008) Hybrid intelligent path planning for articulated rovers in rough terrain. *Fuzzy Sets Syst* 159:2927–2937. doi:10.1016/j.fss.2008.01.029
2. Dolgov D, Thrun S, Montemerlo M, Diebel J (2010) Path planning for autonomous vehicles in unknown semi-structured environments. *Int J Rob Res* 29:485–501. doi:10.1177/0278364909359210
3. Jan GE, Chang KY, Parberry I (2008) Optimal path planning for mobile robot navigation. In: *IEEE/ASME transactions on mechatronics* vol 13, pp 451–460. doi:10.1109/TMECH.2008.2000822
4. Zhang T, Zhu Y, Song JY (2010) Real-time motion planning for mobile robots by means of artificial potential field method in unknown environment. *Ind Rob* 37:384–400. doi:10.1108/01439911011044840
5. Mabrouk MH, McInnes CR (2008) Solving the potential field local minimum problem using internal agent states. *Rob Auton Syst* 56:1050–1060. doi:10.1016/j.robot.2008.09.006

6. Masoud AA (2007) Decentralized self-organizing potential field-based control for individually motivated mobile agents in a cluttered environment: a vector-harmonic potential field approach. *IEEE Trans Syst Man Cybern* 37:372–390. doi: [10.1109/TSMCA.2007.893483](https://doi.org/10.1109/TSMCA.2007.893483)
7. Ren J, McIsaac KA, Patel RV, Peters TM (2007) A potential field model using generalized sigmoid functions. *IEEE Trans Syst Man Cybern* 37:477–484. doi: [10.1109/TSMCB.2006.883866](https://doi.org/10.1109/TSMCB.2006.883866)
8. Peng XG, Xu DM (2012) Intelligent online path planning for UAVs adversarial environments. *Int J Adv Rob Syst* 9:1–12. doi: [10.5772/45604](https://doi.org/10.5772/45604)
9. Antonelli G, Chiaverini S, Fusco G (2007) A fuzzy-logic-based approach for mobile robot path tracking. *IEEE Trans Fuzzy Syst* 15:211–221. doi: [10.1109/TFUZZ.2006.879998](https://doi.org/10.1109/TFUZZ.2006.879998)
10. Son C (2011) Intelligent robotic path finding methodologies with fuzzy/crisp entropies and learning. *Int J Rob Autom* 26:323–336. doi: [10.2316/Journal.206.2011.3.206-3511](https://doi.org/10.2316/Journal.206.2011.3.206-3511)
11. Ng J, Bräunl T (2007) Performance comparison of Bug navigation algorithms. *J Intell Rob Syst Theory Appl* 50:73–84. doi: [10.1007/s10846-007-9157-6](https://doi.org/10.1007/s10846-007-9157-6)
12. Lumelsky VJ, Stepanov AA (1987) Path-planning strategies for a point mobile automaton moving amidst unknown obstacles for arbitrary shape. *Algorithmica* 2:403–430. doi: [10.1007/BF01840369](https://doi.org/10.1007/BF01840369)
13. Lumelsky VJ, Skewis T (1990) Incorporating range sensing in the robot navigation function. *IEEE Trans Syst Man Cybern* 20:1058–1068. doi: [10.1109/21.59969](https://doi.org/10.1109/21.59969)
14. Kamon I, Rivlin E (1997) Sensory-based motion planning with global proofs. *IEEE Trans Rob Autom* 13:814–822. doi: [10.1109/70.650160](https://doi.org/10.1109/70.650160)
15. Kamon I, Rimón E, Rivlin E (1998) TangentBug: a range-sensor-based navigation algorithm. *Int J Rob Res* 17:934–953. doi: [10.1177/027836499801700903](https://doi.org/10.1177/027836499801700903)
16. Guruprasad KR (2012) EgressBug: a real time path planning algorithm for a mobile robot in an unknown environment. *Lect Notes Comput Sci* 7135:228–236. doi: [10.1007/978-3-642-29280-4_27](https://doi.org/10.1007/978-3-642-29280-4_27)
17. Choset H, Lynch KM, Hutchinson S, Kantor G, Burgard W, Kavraki LE, Thrun S (2005) *Principles of robot motion-theory: algorithms, and implementation*. MIT Press, Cambridge